BIGCHALKBOX

EXAMINER SUMMARY SHEET

CSS2257M_QUESTION PAPER | CSS2257M_MT302135

Submitted — · Evaluated 5 Nov 2026

QUESTION-WISE MARKS SUMMARY

Q.NO	QUESTION	MAX	AWARDED
Q1	Explain briefly why the Program Counter (PC) is incremented during the...	2	0.0
Q2	List the five functional units of a digital computer.	2	0.0
Q3	Compare between a Single Bus and a Multiple Bus organization.	2	1.0
Q4	Define Cache Memory and state its purposes.	2	0.5
Q5	Define stack and Stack Pointer (SP).	2	0.5
Q6	a) Explain the basic performance equation of a computer and identify t...	10	2.5
Q9	Represent and in -bit two's complement form and perform the correspond...	10	1.0
Q11	Explain the internal organization of memory chip with a neat diagram.	10	0.0
Q13	a) Explain the PUSH and POP operations on a stack with a neat diagram,...	10	0.0
Total		**50**	**5.5**

Total Marks: 5.5 / 50 (11%)

Overall score: 11.0%. Needs significant improvement in core concepts.

QUESTION 1

COPY

PART - A

1, The program counter (PC)

✗ It can store data and share the of network to computer

Not why PC increments

[2] No fetch-next idea
Missing auto-fetch of next instruction

2 072 Pull the [illegible] instructions of

The answer confuses the Program Counter with a network device, claiming it 'stores data and shares bits of network.' This is unrelated to the fetch cycle. Both required points are absent: that incrementing the PC makes it hold the next instruction's address, and that this lets the processor automatically fetch the following instruction. Define the PC's role in fetch-decode-execute before answering.

QUESTION 1 — RUBRIC-WISE FEEDBACK

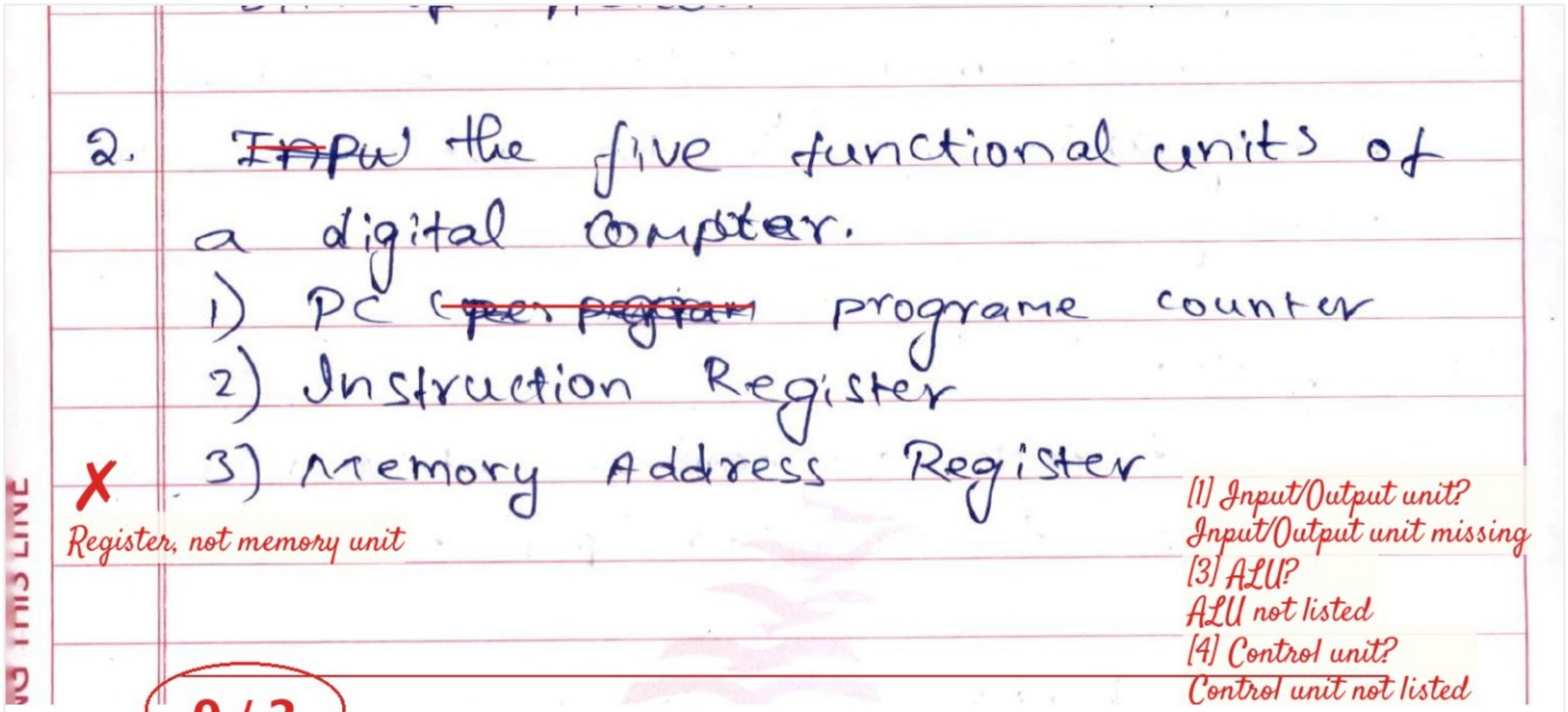
0.0 / 2 marks

CRITERION	MARKS
The PC is incremented during fetch so that it points to the address of the next instruction in sequence *The student never states that incrementing the PC makes it hold the address of the next instruction. Instead the answer describes the PC as something that 'stores data and shares the bits of network to computer', which is irrelevant to the fetch cycle and shows no understanding of the PC's role.*	✕ 0 / 1
ensuring that after the current instruction is fetched, the processor automatically moves on to fetch the following instruction (unless a branch alters it) *There is no mention of the processor automatically moving on to fetch the following instruction after the current one. The written content does not address sequential fetching at all, so this criterion is unmet.*	✕ 0 / 1

FEEDBACK

The response is off-topic: it describes the PC as storing data and sharing network bits rather than explaining why it is incremented during fetch. Neither rubric point (pointing to the next instruction's address, and enabling automatic sequential fetching) is addressed, so no marks can be awarded. The student should revisit the fetch-decode-execute cycle and the specific function of the Program Counter.

QUESTION 2



0 / 2

The five functional units (input, output, memory, ALU, control) were not listed. The answer instead names CPU registers—program counter, instruction register, memory address register—which are internal components, not top-level units. 'Input' in the heading is the question, not an answer item. Next time, recall the standard five-unit model and name each unit directly.

QUESTION 2 — RUBRIC-WISE FEEDBACK

0.0 / 2 marks

CRITERION	MARKS
Input unit, Output unit *The numbered answer list contains no input or output unit. The word 'Input' appears only in the restated question heading (and is partly struck through), not as a listed functional unit, so no credit is given.*	✕ 0 / 0.5
Memory unit *The student wrote 'Memory Address Register', which is a CPU register, not the memory unit. This is a different concept and earns no mark for the memory unit.*	✕ 0 / 0.5
Arithmetic and Logic Unit (ALU) *The ALU is not mentioned anywhere in the answer; the student listed registers (PC, IR, MAR) instead.*	✕ 0 / 0.5
Control unit *The control unit is not listed. The student's three items are all internal registers, none of which correspond to the control unit.*	✕ 0 / 0.5

FEEDBACK

The student misunderstood the question and listed CPU registers (PC, IR, MAR) rather than the five functional units of a digital computer. None of the required units—input, output, memory, ALU, control—appear as answer items, so no marks are awarded. Review the standard five-unit architecture and list each unit by name.

QUESTION 3

CUT ALONG THIS

3. Compare Between Single Bus and a Multiple Bus:-

Single Bus	Multiple Bus
i) It can operates only one operation ✓	i) It can operate multiple operations. ✓
ii) It is only one direction	ii) It is flexible Operation

[2] cost/bottleneck? Missing simple/low-cost/bottleneck
[4] faster/complex? Missing faster/more complex/costlier

1 / 2

The single-vs-multiple transfer distinction is captured (one operation vs multiple operations), the core of the comparison. But the cost and speed trade-offs are missing: single bus is simple and low-cost yet a bottleneck; multiple bus is faster but more complex and costlier. 'One direction' and 'Flexible' do not cover these. Pair each transfer point with its cost and speed consequence.

QUESTION 3 — RUBRIC-WISE FEEDBACK

1.0 / 2 marks

CRITERION	MARKS
Single Bus: one common bus shared by CPU, memory and I/O; only one transfer possible at a time *The student's 'It can operates only one operation' captures the single-transfer-at-a-time behaviour, which is the defining feature of a single bus. The shared CPU/memory/I/O bus is not named, but lenient full credit is given for the core idea.*	✓ 0.5 / 0.5
simple, low cost, but creates bottleneck/lower speed *The 'only one direction' point does not address simplicity, low cost, or the bottleneck/lower-speed drawback; none of these rubric ideas appear anywhere in the single-bus column.*	✕ 0 / 0.5
Multiple Bus: separate/dedicated buses (e.g. address, data, control or multiple internal buses) allow simultaneous transfers *'It can operate multiple operations' captures the simultaneous/multiple-transfer advantage of a multiple-bus organisation. The separate/dedicated buses are not named, but lenient full credit is given for the core idea.*	✓ 0.5 / 0.5
faster but more complex and costlier *'flexible operation' does not mention the greater speed or the increased complexity/cost of multiple buses; the rubric trade-off point is absent.*	✕ 0 / 0.5

FEEDBACK

The answer correctly contrasts the number of simultaneous transfers (single = one operation, multiple = several), which is the central idea of the comparison. It omits the cost/complexity and speed/bottleneck trade-offs that complete the picture, and the 'one direction'/'flexible' points do not map to the rubric. Adding the simple/low-cost/bottleneck versus faster/complex/costlier pairs would secure the remaining marks.

QUESTION 4

4. cache memory:

It's work is remove the data from the memory.

purpose:

cache memory is use to increase the data memory and increase the data memory

Cache stores copies, not removes

[1] missing: high-speed, CPU-main memory
Key definition elements absent
Wrong purpose: no speed/access time

0.5 / 2

5. The definition and purpose are both incorrect. Cache does not remove or clear data, nor increase storage; it is a small high-speed memory between the CPU and main memory holding copies of frequently used data, to cut access time and bridge the speed gap. Anchor the definition on 'fast copy of recent data' and the purpose on speed, not capacity.

QUESTION 4 — RUBRIC-WISE FEEDBACK

0.5 / 2 marks

CRITERION	MARKS
Cache Memory: a small, high-speed memory placed between the CPU and main memory that stores copies of frequently/recently used instructions and data *The definition 'cache remove the data from the memory' is wrong in mechanism (cache stores copies of data; it does not remove it) and omits every key attribute (small, high-speed, between CPU and main memory, frequently/recently used). A lenient 0.5 is given only for the correct high-level association that cache is a memory dealing with data.*	✕ 0.5 / 1
Purpose: to reduce the average memory access time and bridge the speed gap between the fast CPU and slower main memory *The stated purpose 'clear the memory and increase the data storage in memory' is incorrect; cache neither clears memory nor adds capacity. No element of the required purpose (reduce average access time / bridge the CPU-main memory speed gap) appears, so no marks are awarded.*	✕ 0 / 1

FEEDBACK

The student attempted both the definition and the purpose but described cache's function incorrectly, claiming it removes/clears data and increases storage. The correct ideas—that cache is a fast memory holding copies of frequently used data to reduce access time and bridge the CPU-main memory speed gap—are absent. Only a token partial credit is given for relating cache to data in memory.

QUESTION 5

0

5. stack :

It stack ~~operate~~ means It is memory we have to push or pop at one side only

stack pointer :

[2] SP definition?
Term only, no explanation given

0.5 / 2

The stack description captures the single-end push/pop behaviour, which hints at LIFO, but it omits the linear-structure framing and any typical uses. The stack pointer is named only, with no statement that it is a register holding the top-of-stack address. Next time, give a one-line definition for each term and add the LIFO keyword and one use for the stack.

QUESTION 5 — RUBRIC-WISE FEEDBACK

0.5 / 2 marks

CRITERION	MARKS
Stack: a linear data structure that stores data in LIFO (Last-In-First-Out) order, used for subroutine linkage, temporary storage, parameter passing *The student's line 'we have to push or pop at one side only' captures the single-end access idea that underlies LIFO, showing partial understanding. However, the answer never states that a stack is a linear data structure, does not use the LIFO keyword, and gives no typical uses (subroutine linkage, parameter passing), so only half credit is awarded.*	✕ 0.5 / 1
Stack Pointer (SP): a register that holds the address of the top of the stack *The student only wrote the term 'Stack pointer' with no accompanying definition. There is no mention that SP is a register or that it holds the address of the top of the stack, so no credit can be given for this part.*	✕ 0 / 1

FEEDBACK

The student shows a partial grasp of the stack through the 'push or pop at one side only' idea, but the definition is incomplete (no LIFO keyword, no linear-structure label, no uses) and the stack pointer is left entirely undefined. Only the term 'Stack pointer' is written, so the second mark is lost. Overall a weak, under-developed answer worth half a mark.

QUESTION 6

PART-B

6.

a) The basic performance equation of a computer

CPU execution time = clock time x mxnx10^6

It affecting factors are the

clock time $t = \frac{1}{f}$

f = frequency

m = cycles

n = executed times

[4] no conclusion line
Missing performance conclusion

b) Given

L = 100

m = 49

n = 2 F = 2GHz

Step one :-

$t = \frac{1}{f}$

$= \frac{1}{2 \times 10^9} = 0.5 \times 10^{-6}$

[6] no total IC step
Instruction count not found

[7] no total cycles
Cycle total missing
wrong: should be 10^-9

Step 2

Program execution = 100 $0.5 \times 10^{-6} \times 49 \times 2 \times 10^6$

cpu execution time = 49 s

wrong final answer

2.5 / 10

Part (a) gives the equation in non-standard symbols and never defines CPI or instruction count, so the three-factor explanation is absent. Part (b) omits the total-instruction and total-cycle steps, miscomputes the clock period, and yields 49 s instead of 5 us. State IC, CPI and clock rate, then find total cycles before dividing by frequency.

QUESTION 6 — RUBRIC-WISE FEEDBACK

2.5 / 10 marks

CRITERION	MARKS
a) Performance measured by CPU execution time; basic equation CPU Execution Time = (Instruction Count x CPI) / Clock Rate *The student wrote the heading and an equation 'CPU execution time = clock time x m x n x 10^6' with clock time = 1/f, which is structurally equivalent to (IC x CPI)/f but uses undefined symbols and an odd 10^6 factor, and never states that performance is measured by CPU time. Partial credit for the correct underlying structure.*	✕ 1 / 2
a) Definitions of CPI and Clock Rate *The student defined f = frequency (clock rate) and m = cycles (CPI-like), but did not give the proper definitions of CPI as average cycles per instruction or clock rate as cycles per second in Hz.*	✕ 0.5 / 1
a) Three factors (Instruction Count, CPI, Clock Rate) and how each affects time *The student listed the affecting factors (clock time/frequency, cycles, number of times) but only as a list; there is no explanation that fewer instructions, lower CPI or higher clock rate reduce execution time.*	✕ 0.5 / 1.5
a) Conclusion that reducing IC/CPI or increasing clock rate improves performance *No concluding sentence linking the factors to improved performance was written.*	✕ 0 / 0.5
b) Listing the given values *A 'given' section lists L = 100, m = 49, n = 2, F = 2 GHz, capturing most of the data, though the separate loop-body instruction count of 100 is not clearly stated.*	✓ 0.5 / 0.5
b) Step 1: total instructions = 100 + 100x49 = 5000 *The student's 'Step one' computes the clock period t = 1/f instead of the total instruction count; the 5000-instruction total is never derived.*	✕ 0 / 1.5
b) Step 2: total clock cycles = 5000 x 2 = 10000 *No total-cycle computation appears; the value 2 is only listed as n and never multiplied into a cycle total.*	✕ 0 / 1
b) Step 3: execution time = 10000 / (2x10^9) = 5x10^-6 s *The clock period was miscomputed as 0.5x10^-6 s (should be 0.5x10^-9 s) and the final time was obtained by an incorrect multiplication rather than dividing total cycles by the clock rate.*	✕ 0 / 1.5
b) Final answer = 5 microseconds *The student's final answer is '49 s', which is incorrect; the correct value is 5 microseconds.*	✕ 0 / 0.5

FEEDBACK

The student shows a partial grasp of the performance equation in part (a) but writes it with undefined symbols and omits the factor explanations and conclusion. Part (b) is largely incorrect: the total-instruction and total-cycle steps are skipped, the clock period is miscalculated, and the final answer (49 s) is far from the correct 5 microseconds. Overall the method needs to follow IC, CPI and clock rate explicitly before computing time.

9) a)

Step - 1 :-

Convert positive +47 to Binary

✓ . $(1011111)_2$

1's Complement :-

0100000

2's Complement :-

1011111

for negative add 1

11011111

-47 wrong (need 11010001)

Convert 25 to Binary

✓ $(0110001)_2$

1's Complement :-

0100110

2's complement

0011001

for positive add 0

00011001

not 8-bit

[3] -25 missing?
-25 never formed

operation :-

11011111

(-) 00011001

[4] no sum?
operation left unfinished

[5] verify -72?
no interpretation given

[6] overflow?
no overflow comment

b) Step-1 :

Convert positive +100 to Binary

$(1100100)_2$

for negative add 1

11100100

2's complement 11100100

convert positive 50 to Binary

$(01100010)_2$

for negative, add 1

10111000 10

operation :-

~~1+100100~~

(+) ~~00110010~~

~~a 1+1001100~~ 11100100

(+) ~~a 00+0001~~0 (+) 10110010

~~1+0000010~~ 10010110

~~(10000110)~~

answer :- 10011110

(-) -150

1 / 10

Part (a) is incomplete: the 8-bit 2's-complement forms of -47 and -25 were not obtained (1101111 and a 7-bit +25 are wrong), the subtraction was never turned into addition of -25, and no sum, verification, or overflow comment appears. Use 8-bit patterns throughout, convert subtraction to addition of the 2's-complement, then check carry-out and sign bits for overflow.

QUESTION 9 — RUBRIC-WISE FEEDBACK

1.0 / 10 marks

CRITERION	MARKS
47 = 00101111. –47 in 8-bit 2's complement = 11010001 (invert + 1) *The student converted 47 to binary (101111) and showed a 1's-complement / add-1 step, but used 6/7-bit patterns and the final –47 value 1101111 is incorrect (expected 8-bit 11010001), so only partial credit is given.*	✕ 0.5 / 2
+25 = 00011001 *25 was correctly converted to binary (011001), but the final +25 pattern written (0011001) is 7-bit rather than the required 8-bit 00011001.*	✕ 0.5 / 1
–25 in 2's complement = 11100111 *The student never formed –25 in 2's complement; they kept +25 and set up a subtraction instead of converting the operation to addition of –25.*	✕ 0 / 1
(–47)–(+25) = (–47)+(–25): 11010001 + 11100111 = 1 10111000 → discard carry-out → result = 10111000 *No addition of the two 2's-complement operands was carried out; the operation line 1101111 – 0011001 was left unfinished with no carry-out handling and no result.*	✕ 0 / 3
Interpreting 10111000 as signed 8-bit = –72, which matches (–47–25 = –72) *Because no result was produced for part (a), there is no interpretation as –72 and no verification against –47 – 25.*	✕ 0 / 2
No overflow occurs because although both operands are negative, the magnitude of the true result (72) still fits within the 8-bit signed range (–128 to +127) *No comment on overflow for part (a) was given anywhere in the answer.*	✕ 0 / 1

FEEDBACK

The student attempted both parts but the work scored here (part a) is largely incorrect and unfinished: binary patterns are 6/7-bit rather than 8-bit, the 2's-complement of –47 is wrong, –25 was never formed, and the operation was set up as a subtraction that was never completed, leaving no result, verification, or overflow comment. Only the correct decimal-to-binary magnitudes of 47 and 25 earn partial credit.

QUESTION 11

18)

The Internal organization of 16x4 memory Chip Is 64 ~~bit~~ byte It will store ~~consists~~ ~~data~~ 8 bits of 8 words of 64 byte

[1] Register indirect?
No EA=2000 / M[2000]=500
[2] Indexed EA?
No R1+50=2050
[3] Auto-increment?
No R1 becomes 2002

0 / 10

The note on a 16x4 chip (64 bytes, 8-bit words) does not address the required content. No effective-address computation for register indirect, indexed, or auto-increment appears, and no diagram is drawn. Next time, work out EA = R1, R1+50, and the post-increment value, and sketch the chip's address and data lines.

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

CUT ALONG THIS LINE

QUESTION 11 — RUBRIC-WISE FEEDBACK

0.0 / 10 marks

CRITERION	MARKS
(i) Register Indirect: EA = contents of R1 = 2000; operand = M[2000] = 500 *The student's text only describes a 16x4 chip as 64 bytes of 8-bit words; there is no register-indirect effective-address computation (EA = R1 = 2000, operand = M[2000] = 500) anywhere.*	✕ 0 / 3
(ii) Indexed with displacement 50: EA = R1 + 50 = 2000+50 = 2050; operand = M[2050] (whatever value is stored there) *No indexed addressing is shown; the student never forms EA = R1 + 50 = 2050 or references M[2050].*	✕ 0 / 3
(iii) Auto-increment: EA = current value of R1 = 2000; operand fetched = M[2000] = 500; after access, R1 is incremented by the operand size (2 bytes) → R1 becomes 2002 *The auto-increment behaviour (R1 going from 2000 to 2002 after fetching M[2000]) is entirely absent from the answer.*	✕ 0 / 4

FEEDBACK

The answer only states the size of a 16x4 memory chip and contains no diagram. None of the required addressing-mode effective-address calculations (register indirect, indexed, auto-increment) are present, so no rubric criterion is met. The crossed-out words were ignored in scoring.

13) a) ⇄

The push and pop operations on a stack with a neat diagram :-

PUSH :-

It can operates the Push (or) ~~del~~ Remove the ~~array from the~~ ele, from the array

PUSH wrongly = remove

POP :-

It can operates the ~~add~~ (or) insert the array of elements from the array.

POP wrongly = insert

example

[1] No diagram / SP? Stack diagram and SP missing

CUT ALONG THIS LINE

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

b) The steps involved in the calculating of 5 array elements :-

add R1, R2

$R_3 \rightarrow R_1 + R_2$

add R3, R4

$t_2 \rightarrow R_3 + R_4$

add R4, R5 t1, t2

$t_3 \rightarrow t_1 + t_2$

add t3, R5

$t_4 \rightarrow t_3 + R_5$

destination :-

$t_4 = R_1 + R_2 + R_3 + R_4 + R_5$

10

[4] No counter (Move 5,R1)
Counter missing
[5] No list pointer
Pointer missing
[6] No CALL
Subroutine call missing
[7] No save to SUM
Result not stored
[8] No Clear R0
Sum not initialised
[9] No (R2)+ add
Looped add missing
[10] No decrement
Counter decrement missing
[11] No branch
Loop branch missing
[12] No RETURN
Return missing

0 / 10

Part (a) inverts PUSH and POP and never references the stack pointer; no diagram is drawn, so the SP-decrement/SP-increment behaviour is unaddressed. Part (b) is a flat chain of register adds with no CALL, loop, counter, or RETURN, so the subroutine and register-parameter-passing requirements are entirely missing. Tip: state SP movement explicitly with a downward-growing stack diagram, and structure (b) as Move counter, Move pointer, CALL, looped (R2)+ add, decrement, branch, RETURN.

QUESTION 13 — RUBRIC-WISE FEEDBACK

0.0 / 10 marks

CRITERION	MARKS
a) Diagram should show the stack growing downward in memory with SP moving down on PUSH and up on POP *No diagram is drawn at all; the student only writes the phrases 'with a neat diagram' and 'example' with no stack, memory, or SP illustration following them.*	✕ 0 / 2.5
a) PUSH: SP is decremented (stack grows downward) and the operand is stored at the address now pointed to by SP. *The PUSH description is inverted (it says push removes elements) and there is no mention of decrementing SP or storing the operand at the SP address.*	✕ 0 / 1.25
a) POP: the operand is read from the address pointed to by SP, and then SP is incremented. *The POP description is inverted (it says pop adds/inserts elements) and there is no mention of reading at SP then incrementing SP.*	✕ 0 / 1.25
b) Move 5, R1 ; R1 serves as a counter *No counter register is loaded with the value 5 anywhere in the answer.*	✕ 0 / 0.5
b) Move #NUM1, R2 ; R2 points to first element of the list *No pointer register is set to the address of the first array element; R1-R5 are treated as the values themselves.*	✕ 0 / 0.5
b) Call LISTADD ; Call subroutine *There is no CALL instruction and no subroutine is invoked; the work is done inline.*	✕ 0 / 0.5
b) Move R0, SUM ; Save result *The final sum is left in a temporary (t4) and never stored to a SUM memory location.*	✕ 0 / 0.5
b) LISTADD Clear R0 ; Initialize sum to 0 *No accumulator/sum register is cleared or initialised to zero before adding.*	✕ 0 / 0.5
b) LOOP Add (R2)+, R0 ; Add entry from list *The additions are between plain registers (R1+R2, etc.), not a looped (R2)+,R0 accumulation from memory.*	✕ 0 / 1
b) Decrement R1 *No counter decrement instruction appears.*	✕ 0 / 0.5
b) Branch>0 LOOP *No conditional branch back to a loop is present.*	✕ 0 / 0.5
b) Return ; Return to calling program *No RETURN instruction is written, consistent with the absence of any subroutine.*	✕ 0 / 0.5

FEEDBACK

Both parts miss the mark. Part (a) describes PUSH as removing and POP as inserting (the reverse of correct), never mentions the stack pointer, and contains no diagram. Part (b) is a sequential set of register additions with no subroutine, CALL, RETURN, loop, counter, or parameter passing, so none of the required steps are present. Revisit stack SP behaviour and the standard CALL/LOOP/RETURN subroutine pattern.